

NEON SNAKE — Combined Documentation

Daily Game Development Series, Day 5 — Whitepaper · History · Source Code Detail

Project	NEON SNAKE
Series	Daily Local Game Development Series — Day 5
Author	Mayank Swaraj
Brand	Mayank Creations
Stack	Vanilla JavaScript, HTML5 Canvas, Web Audio API
Document Type	Combined Whitepaper + History + Source Code Detail

WHITEPAPER

1. Overview

NEON SNAKE is Day 5 of the daily vanilla JavaScript game development series under Mayank Creations. It is a grid-based snake game: a player-steered segment chain grows by consuming food items, with self-collision ending the run. A secondary golden food item spawns probabilistically and awards triple score, adding an optional risk layer on top of the core survival loop.

Prior entries: CRATE PUSH (DOM grid puzzle), NEON DODGER (canvas vertical dodge), GRAVITY FLIP (canvas side-scroll runner), SHARD BREAK (canvas brick-breaker). Day 5 reintroduces a grid model — as in Day 1 — but replaces static puzzle logic with a real-time tick-based simulation and a growing entity with self-collision semantics.

2. Design Goals

- Tick-based movement over frame-rate-tied movement: the snake advances one cell per configurable interval, keeping gameplay feel consistent across device frame rates.
- Wrapping grid: the snake passes through walls to the opposite edge, removing wall-collision as a death source and focusing all risk on self-intersection.
- Dual food system: standard food grows the snake by one cell; golden food (star shape, 25% spawn chance after each fifth food) awards 30 points versus 10, creating a risk-reward decision

about pursuit.

- Speed escalation every five foods eaten: stepMs decreases by 18ms per tier to a floor of 60ms, providing a natural difficulty ramp without discrete level transitions.

3. Core Mechanics

The snake is a JavaScript array of coordinate objects. Each tick, a new head cell is prepended by adding the current direction vector to the head position modulo grid dimensions. If the head occupies any segment from index 1 onward, the run ends. If the head coincides with a food cell, the tail is not removed (net growth of one cell). Otherwise the tail is popped (net length unchanged). This prepend/pop model avoids shifting or copying the entire array on each tick.

Direction input is double-buffered: the current direction is applied at tick time from a nextDir buffer, and a 180-degree reversal guard prevents the snake from immediately self-colliding by steering into its own neck.

4. Rendering

Each snake segment is drawn as a filled rectangle with a two-pixel inset gap creating a segmented appearance. Segment colour interpolates from bright green at the head through a darkening gradient toward the tail, making length visually apparent. Eyes are drawn on the head segment offset by the current direction vector and its perpendicular. Food is a filled circle with a glow shadow; golden food is a five-pointed star constructed from paired arc angles. A faint grid overlay reinforces the discrete cell structure without competing with gameplay elements.

5. Audio Design

- Eat: sine 440 Hz, gives tactile confirmation without the harsh quality of square waves at higher frequencies.
- Gold eat: two ascending sine tones (660 Hz then 880 Hz) 70ms apart, distinguishable from standard eat.
- Turn: very short quiet square pulse — subtle enough not to fatigue on rapid direction changes.
- Death: two-note descending sawtooth, consistent with the death sound language established on Days 2, 3, and 4.
- Start: single square confirm tone, matching the series convention.

HISTORY PAPER

1. Position in the Series

NEON SNAKE is Day 5. Series history: CRATE PUSH (grid, turn-based, DOM), NEON DODGER (canvas, real-time, vertical), GRAVITY FLIP (canvas, real-time, side-scroll, one-input), SHARD BREAK (canvas, real-time, physics, multi-object). Day 5 returns to a grid model but operates it in real-time via a fixed tick loop, distinguishing it from the turn-based stepping of Day 1.

2. Build Timeline

Built as a single consolidated pass. The primary design decision resolved during the build was the tick accumulator pattern: rather than using `setInterval` (which drifts under tab-background throttling), the game loop runs on `requestAnimationFrame` and accumulates elapsed delta time, advancing the snake by one tick whenever the accumulator exceeds the current step interval. This keeps the snake speed accurate regardless of frame-rate variation.

Documentation produced immediately as one combined PDF per the standing series format.

3. Decisions and Trade-offs

- Wrapping grid over bounded grid: removes wall death, concentrating all player skill on path planning and self-avoidance — more appropriate for a mobile swipe interface where precision steering is harder.
- Array prepend/pop over ring buffer: simpler and correct at snake lengths below a few hundred cells, which covers all realistic play sessions.
- `nextDir` buffer over immediate direction application: prevents the well-known snake bug where two rapid inputs within one tick cause a 180-degree reversal.
- Golden food as probabilistic spawn (25%) rather than fixed interval: keeps the golden food appearance feeling like a bonus event rather than a predictable resource.
- `stepMs` floor at 60ms: corresponds to roughly one cell per frame at 60fps — below this the snake moves faster than a player can react via swipe input.

SOURCE CODE DETAIL

1. Tick Accumulator

The game loop accumulates delta time and advances the snake only when the accumulator exceeds the current step interval, making snake speed frame-rate independent.

```
stepAcc += dt;
if (stepAcc >= stepMs) {
  stepAcc -= stepMs;
  step();
}
```

2. Wrapping Head Calculation

```
var head = {  
  x: (snake[0].x + dir.x + COLS) % COLS,  
  y: (snake[0].y + dir.y + ROWS) % ROWS  
};
```

3. Self-Collision Check

Iterates from index 1 (not 0) to avoid comparing the new head against the old head position.

```
for (var i = 1; i < snake.length; i++) {  
  if (snake[i].x === head.x && snake[i].y === head.y) {  
    die(); return;  
  }  
}
```

4. Direction Guard

```
function tryDir(dx, dy) {  
  if (dx === dir.x * -1 && dy === dir.y * -1) return;  
  nextDir = { x: dx, y: dy };  
  sndTurn();  
}
```

5. Free Cell Search

Randomised rejection sampling with a 500-attempt cap. At large snake lengths (approaching grid capacity) this can become slow; a shuffled-index approach would be more efficient but is unnecessary at practical play lengths on an 18x24 grid.

```
function freeCell(body) {  
  var occ = {};  
  body.forEach(function(s) { occ[s.x + ',' + s.y] = 1; });  
  var attempts = 0;  
  while (attempts++ < 500) {  
    var x = rnd(COLS), y = rnd(ROWS);  
    if (!occ[x + ',' + y]) return { x: x, y: y };  
  }  
  return null;  
}
```

6. Golden Food Star Render

Five outer vertices and five inner vertices at $0.42\times$ radius, alternating, producing a five-pointed star without external geometry libraries.

```
for (var s = 0; s < 5; s++) {  
  var a1 = s * Math.PI * 2 / 5 - Math.PI / 2;
```

```
var a2 = (s + 0.5) * Math.PI * 2 / 5 - Math.PI / 2;  
cx.lineTo(gx + Math.cos(a1)*gr, gy + Math.sin(a1)*gr);  
cx.lineTo(gx + Math.cos(a2)*gr*0.42, gy + Math.sin(a2)*gr*0.42);  
}
```

7. Known Constraints

- freeCell uses rejection sampling — degrades near grid saturation (>80% fill).
- Best score is session-only; no localStorage persistence.
- No wall-collision death; wrapping is intentional for mobile-swipe ergonomics.

"Every system I build is a brick in a larger, self-taught architecture of independent AI work."

- Mayank Swaraj